

Consumindo APIs com HTML, CSS e JavaScript

Do primeiro fetch() a um buscador de endereços completo, usando as APIs públicas do ViaCEP e do IBGE

Três etapas, do conceito ao código

1

Fundamentos

O que é uma API, como o HTTP funciona e como ler JSON.

cerca de 30 min

2

JavaScript para APIs

Assincronia, Promises, fetch, async/await e o DOM.

cerca de 50 min

3

Projeto prático

Explorador de Endereços com as APIs do ViaCEP e do IBGE.

cerca de 70 min

Meta: sair da aula com um buscador de endereços funcionando.

Parte 1: fundamentos

Uma API funciona como o garçom de um restaurante



API (Application Programming Interface) é um contrato: diz o que você pode pedir, como pedir e o que vai receber. Você não precisa saber como a cozinha funciona, basta seguir o cardápio, que é a documentação.

Parte 1: fundamentos

Toda requisição tem um endereço e um método

https://	viacep.com.br	/ws/	01001000	/json/
protocolo	domínio	rota	parâmetro (o CEP)	formato

Método	Para que serve	Exemplo
GET	buscar dados	consultar um CEP
POST	criar algo novo	cadastrar um usuário
PUT / PATCH	atualizar	editar um perfil
DELETE	remover	apagar um comentário

Nesta aula usamos só GET: as duas APIs do projeto são de leitura.

A resposta chega com um código de status

2xx

Deu certo

200 OK: os dados vieram

201 Created: item criado

4xx

Erro do cliente

400 Bad Request: pedido mal formatado

404 Not Found: recurso não existe

5xx

Erro do servidor

500: falha interna da API

503: serviço fora do ar

No ViaCEP, um CEP com letras ou com tamanho errado devolve 400. Vamos tratar esse caso no projeto.

Parte 1: fundamentos

JSON: o formato em que os dados viajam

Resposta do ViaCEP para 01001-000 (resumida)

```
{  
  "cep": "01001-000",  
  "logradouro": "Praça da Sé",  
  "complemento": "lado ímpar",  
  "bairro": "Sé",  
  "localidade": "São Paulo",  
  "uf": "SP",  
  "ibge": "3550308",  
  "ddd": "11"  
}
```

Parece objeto, mas é texto

As chaves ficam sempre entre aspas duplas.

Poucos tipos

Textos, números, true e false, null, listas [] e objetos { }.

Vira objeto no JS

resposta.json() converte o texto. Depois, dados.bairro vale "Sé".

Parte 2: JavaScript para APIs

Buscar dados na rede leva tempo. O JavaScript não fica parado esperando. esperando.

Por isso, o código que conversa com uma API é assíncrono: faz o pedido, segue executando e trata a resposta quando ela chegar.

Parte 2: JavaScript para APIs

Síncrono e assíncrono: a ordem muda

Síncrono: uma coisa por vez

```
console.log("1. Pedi a pizza");  
console.log("2. Pizza chegou");  
console.log("3. Vou ver TV");
```

Saída: 1, 2, 3

Assíncrono: a espera fica de lado

```
console.log("1. Pedi a pizza");  
setTimeout(() => {  
  console.log("2. Pizza chegou");  
}, 2000);  
console.log("3. Vou ver TV");
```

Saída: 1, 3 e, 2 s depois, 2

O `fetch()` se comporta como o `setTimeout`: a resposta chega depois.

Promise: a promessa de um valor futuro



`fetch()` e `resposta.json()` devolvem Promises. Usamos `.then()` ou `await` para pegar o valor quando ele chegar.

fetch() com .then(): a primeira requisição

```
fetch("https://viacep.com.br/ws/01001000/json/")
  .then((resposta) => resposta.json())
  .then((dados) => {
    console.log(dados.logradouro);
  })
  .catch((erro) => {
    console.error("Falhou:", erro);
  });
```

- 1 fetch() dispara o pedido e devolve uma Promise.
- 2 resposta.json() converte o corpo em objeto. Também é uma Promise.
- 3 O segundo .then() recebe os dados prontos e mostra Praça da Sé.
- 4 .catch() recebe o erro se a conexão falhar.

Parte 2: JavaScript para APIs

async/await: o mesmo pedido, lido de cima para baixo

```
async function buscarCep(cep) {  
  try {  
    const url = `https://viacep.com.br/ws/${cep}/json/`;  
    const resposta = await fetch(url);  
    if (!resposta.ok) {  
      throw new Error(`Erro HTTP ${resposta.status}`);  
    }  
    const dados = await resposta.json();  
    return dados;  
  } catch (erro) {  
    console.error(erro.message);  
  }  
}
```

async marca a função como assíncrona. Ela sempre devolve uma Promise.

await pausa só esta função até a Promise resolver. A página continua respondendo.

fetch() só rejeita em falha de rede. Para 404 ou 500, confira resposta.ok e lance o erro.

Parte 2: JavaScript para APIs

DOM: onde os dados viram tela

Comando	O que faz	No projeto
<code>document.querySelector("#cep")</code>	encontra um elemento	pegar o campo do CEP
<code>form.addEventListener("submit", fn)</code>	reage a um evento	enviar a busca
<code>campo.value</code>	lê o que foi digitado	o CEP informado
<code>el.textContent = "..."</code>	escreve texto com segurança	mensagens e endereço
<code>el.classList.add("ativo")</code>	liga um estilo do CSS	estado de carregando
<code>document.createElement("li")</code>	cria um elemento novo	lista de municípios

Prefira `textContent` a `innerHTML` para exibir dados vindos de fora: assim nenhum HTML estranho é executado na sua página.

Recursos do JavaScript que vamos usar

const e let

```
const api = "viacep";
```

const para o que não muda; let para o que muda.

Template strings

```
`CEP ${cep} inválido`
```

Monta textos com variáveis dentro de \${ }.

Arrow functions

```
const dobro = (n) => n * 2;
```

Um jeito curto de escrever funções.

Desestruturação

```
const { uf, bairro } = dados;
```

Tira várias propriedades de um objeto de uma vez.

map, filter, forEach

```
estados.map((e) => e.nome)
```

Percorrem listas, como estados e municípios.

Expressão regular

```
cep.replace(/\D/g, "")
```

Remove tudo o que não é número do CEP.

Parte 3: projeto prático

Explorador de Endereços

Digite um CEP e veja o endereço completo. Escolha um estado e liste todos os seus municípios.

ViaCEP

Consulta de CEP

[GET viacep.com.br/ws/{cep}/json/](https://viacep.com.br/ws/{cep}/json/)

Gratuita, sem cadastro e sem chave de acesso.

IBGE Localidades

Estados e municípios

[GET servicodados.ibge.gov.br/api/v1/
localidades/estados/{UF}/municipios](https://servicodados.ibge.gov.br/api/v1/localidades/estados/{UF}/municipios)

Dados oficiais e abertos, também sem chave.

Parte 3: projeto prático

O caminho de um CEP até a tela



⚠ Se qualquer etapa falhar, a tela mostra o que aconteceu e como resolver.

index.html

a estrutura: formulário, campos e áreas de resultado

style.css

a aparência e os estados visuais

script.js

o comportamento: eventos, pedidos e renderização

Parte 3: projeto prático

O coração do projeto: o envio do formulário

```
formulario.addEventListener("submit", async (evento) => {  
  evento.preventDefault();  
  const cep = campoCep.value.replace(/\D/g, "");  
  if (cep.length !== 8) {  
    mostrarErro("Digite um CEP com 8 números.");  
    return;  
  }  
  mostrarCarregando();  
  try {  
    const endereco = await buscarCep(cep);  
    mostrarEndereco(endereco);  
  } catch (erro) {  
    mostrarErro(erro.message);  
  }  
});
```

preventDefault() impede o formulário de recarregar a página.

Validar antes de chamar a API evita pedidos inúteis e responde na hora.

Funções pequenas com nomes claros: cada uma faz uma coisa só.

Parte 3: projeto prático

Uma pegadinha do ViaCEP e os estados da tela

Um CEP bem formatado que não existe volta com status 200 e o corpo abaixo. O `resposta.ok` não pega esse caso:

```
{ "erro": true }
```

```
const dados = await resposta.json();  
if (dados.erro) {  
  throw new Error("CEP não encontrado.");  
}
```

Vazio

convida a digitar um CEP

Carregando

botão desabilitado e texto
Buscando

Sucesso

cartão com o endereço
completo

Erro

mensagem que diz o que fazer

Para praticar

Desafios para depois da aula

Nível fácil

Busca automática

Buscar sozinho quando o campo completar 8 números, sem clicar no botão.

Nível fácil

Ver no mapa

Adicionar um link que abre o endereço encontrado no OpenStreetMap.

Nível médio

Histórico de buscas

Guardar as 5 últimas buscas no localStorage e mostrá-las como atalhos.

Nível difícil

Busca reversa

Descobrir o CEP a partir de UF, cidade e rua:
[viacep.com.br/ws/RS/Porto Alegre/Domingos/json/](https://viacep.com.br/ws/RS/Porto%20Alegre/Domingos/json/)

Encerramento

O que levamos desta aula

- ✓ Uma API é um contrato de pedido e resposta, e o HTTP é a língua que ela fala.
- ✓ `fetch()` devolve uma Promise; `async/await` deixa o código legível.
- ✓ `resposta.ok` e `try/catch` cobrem os erros que o `fetch` sozinho não pega.
- ✓ Toda tela que depende de rede precisa de estados: carregando, sucesso e erro.